

环境 airfogsim_env.py

airfogsim_env.py 是模拟平台的核心环境类，实现了空地协同边缘计算场景的全面仿真功能。

类概述

AirFogSimEnv 类是整个仿真平台的主体，提供了**通信、计算、存储、电池管理、车辆/无人机轨迹、云/边缘节点、AI模型、区块链、认证和隐私**等全方位模拟功能。该类为代理（可以是深度强化学习代理、规则代理或人类玩家）提供了与环境交互的API。

主要组件和属性

实体管理

- vehicles、UAVs、RSUs、cloudServers：管理各类节点实体的字典
- vehicle_ids_as_index、uav_ids_as_index 等：作为索引的实体ID列表
- removed_vehicles、removed_UAVs：已移除实体的记录

配置与时间管理

- config：环境配置参数
- simulation_time：当前仿真时间
- max_simulation_time：最大仿真时间
- simulation_interval：仿真时间步长
- traffic_interval：交通更新间隔（要 $\geq$ simulation_interval）

管理器组件

_configManagersModels

- traffic_manager：交通管理器，负责车辆和UAV的移动
- task_manager：任务管理器，负责任务生成和调度
- channel_manager：信道管理器，负责无线通信
- mission_manager：任务管理器，负责众包感知等任务
- sensor_manager：传感器管理器
- blockchain_manager：区块链管理器
- energy_manager：能源管理器

- node_state_manager: 节点状态信息管理器

决策相关

86-95行

- vehicle_mobility_patterns、uav_mobility_patterns: 移动模式控制
- uav_routes: UAV路由
- new_missions: 新任务列表
- activated_offloading_tasks_with_RB_Nos: 资源分配
- alloc_cpu_callback: CPU分配回调
- task_node_ids: 任务节点ID列表
- task_return_routes: 任务返回路由

指标统计

- 各种通信信道的统计数据: channel、V2U_channel、V2I_channel、U2I_channel

主要函数解析

初始化和配置

init(self, config, interactive_mode=None)

- **功能:** 初始化 AirFogSim 环境的主要组件
- **参数:**
 - config: 环境配置字典, 采用YAML格式
 - interactive_mode: 交互模式, 可选值为'graphic'或'text'
- **详细流程:**
 - 初始化各类实体存储容器 (vehicles, RSUs, UAVs, cloudServers)
 - 设置仿真时间参数
 - 连接SUMO交通仿真器
 - 配置各种管理器 (交通、任务、通信等)
 - 设置任务节点相关配置
 - 初始化决策相关容器和指标

init_indicators(self)

- **功能:** 初始化仿真指标统计数据

- 重置通信信道统计数据（时间和数据大小）

_configManagersModels(self)

- 配置各种管理器，包括交通、任务、通信、任务、传感器、区块链和能源管理器
 - 配置交通管理器并初始化RSU和云服务器
 - 配置任务管理器，预设4秒的可预测时间
 - 配置信道管理器，设置节点数和位置信息
 - 配置任务、传感器、区块链、能源和状态信息管理器

_connectToSUMO(self, config, useSUMO=True)

- 连接到SUMO交通仿真器，设置仿真参数
 - 生成唯一标签
 - 构建SUMO命令行参数
 - 启动SUMO并获取连接对象

mountVisualizer(self, mode='graphic')

- 挂载可视化器，支持图形或文本模式

仿真流程控制

step(self)

- **功能：**执行一个仿真时间步长的全部更新
- **返回值：**仿真是否结束的布尔值
- **详细流程：**
 1. 更新交通状态
 2. 更新AI模型
 3. 更新传感器
 4. 在每个交通步长内执行多个仿真步骤：
 - 更新认证和隐私
 - 更新任务
 - 更新任务（如众包感知）
 - 更新无线通信
 - 更新有线通信
 - 更新计算

- 更新存储
- 更新能源
- 更新区块链

5. 更新仿真时间

6. 记录状态信息

7. 清除决策

reset(self)

- 重置环境至初始状态
 - 关闭现有连接
 - 重置指标数据
 - 重新连接SUMO
 - 依次重置各管理器（交通、任务、通信等）
 - 重置仿真时间和退出标志

isDone(self)

- 检查仿真是否结束的条件, 当仿真时间超过最大时间或强制退出标志为True时返回True

close(self)

- 关闭环境，释放资源

clearDecisions(self)

- 清除当前决策，准备下一个时间步。重置所有决策相关的容器（任务分配、资源分配等）

实体管理

_initRSUsAndCloudServers(self)

- 初始化路边单元RSU和云服务器。
 - 从交通管理器获取RSU和云服务器信息
 - 为每个尚未创建的RSU和云服务器创建实例并存储

_initVehicle(self, vehicle_traffic_info, task_profile={}, fog_profile={})

- 初始化车辆实体

_updateTraffics(self)

更新所有交通相关实体的状态

- 更新交通信息，包括车辆和UAV的位置、速度等
- 处理车辆和UAV的加入和离开
 - 更新车辆和UAV的移动模式
 - 执行交通仿真步进
 - 获取最新的车辆和UAV交通信息
 - 处理新加入的车辆和UAV
 - 更新每个车辆和UAV的状态
 - 处理需要移除的车辆
 - 更新ID索引列表
 - 更新通信管理器的节点信息
 - 检查UAV数量，必要时强制退出

_removeVehicle(self, vehicle_id)

- 安全移除车辆及其相关任务、传感器和任务
 - 从任务节点列表中移除（如果存在）
 - 移除该车辆的所有任务
 - 使该车辆相关的正在执行的任务失败
 - 禁用该车辆的传感器
 - 从索引列表中移除
 - 从新任务列表中移除指向该车辆的任务
 - 删除车辆对象并记录

_removeUAV(self, UAV_id)

- 安全移除UAV及其相关任务、传感器和任务

获取实体信息

_getNodeById(self, node_id)、_getNodeTypeId(self, node_id)、 _getNodeIdxById(self, node_id)

- 通过ID获取节点对象、类型和索引

getVehicleIds(self)、getRSUIds(self)、getUAVIds(self)、getCloudServerIds(self)

- 获取各类实体的ID列表

getVehicleById(self, id)、getRSUById(self, id)、getUAVById(self, id)、
getCloudServerById(self, id)

- 通过ID获取特定实体

_getDistanceBetweenNodes(self, node1, node2)、
getDistanceBetweenNodesById(self, node_id_1, node_id_2)

- 计算节点间距离

任务和通信管理

_updateTask(self)

- 更新和生成任务
 - 获取任务节点的配置
 - 生成并检查任务
 - 设置任务返回路由
 - 最终检查所有任务

_allocate_communication_RBs(self, activated_offloading_tasks_with_RB_Nos)

- 为卸载任务分配通信资源（资源块）
 - 获取正在卸载的任务
 - 重置活跃链路
 - 遍历每个任务，设置发送方和接收方
 - 激活对应的通信链路
 - 构建并返回任务配置字典

_updateWirelessCommunication(self)

- 更新无线通信，处理任务卸载
 - 分配通信资源
 - 计算通信速率
 - 执行通信过程

_compute_communication_rate(self, activated_task_dict)

- 根据信道模型计算通信速率

_execute_communication(self, activated_task_dict)

- 执行通信过程，根据信道速率和SINR传输任务

- 临时存储成功和失败的任务
- 对每个任务执行传输
- 更新通信统计数据
- 设置本时间步的传输大小
- 完成或失败标记相应任务

计算和资源管理

`_updateComputation(self)`

- 更新计算过程，分配CPU资源执行任务: 调用任务管理器的computeTasks方法，使用设置的CPU分配回调函数

`_updateEnergy(self)`

- 更新能源状态，包括电池消耗和充电
 - 遍历所有UAV，更新其能源模式
 - 执行能源更新
 - 处理电量耗尽的UAV移除
 - 更新节点数
 - 如果所有UAV都移除，则强制退出仿真

其他功能

`_updateAIModels(self)`

- 更新AI模型，用于联邦学习、迁移学习等场景

`_updateMission(self)`

- 更新任务，检查位置、执行情况和截止日期

`_updateSensor(self)`

- 更新传感器状态

`_updateStateInfo(self)`

- 更新实体状态信息，记录日志
 - 获取当前的fog node和task node的IDs
 - 获取它们的状态信息
 - 记录节点状态

- 记录任务状态

`payAndPunish(self, revenue_and_punishment_for_tasks)`

- 根据任务执行情况对节点进行奖惩

`getChannelAvgRate(self, channel_type=None)`

- 获取特定类型通信信道的平均速率

主要流程

1. 初始化阶段：创建环境实例，配置各管理器，连接SUMO

2. 仿真循环：

- 更新交通状态（车辆、UAV位置）
- 更新AI模型
- 更新传感器状态
- 更新任务、通信、计算、存储和能源
- 更新区块链
- 记录状态信息

3. 决策执行：

- 外部算法（如DRL）做出决策
- 更新移动模式、任务分配、资源分配
- 执行通信和计算

4. 统计与可视化：

- 计算各类指标
- 可视化当前状态